

Zero Trust Security for Web Applications in Microservice-Based Environments

Jayaraj Viswanathan
Department of Computer Science and Engineering,
Amrita Vishwa Vidyapeetham,
Chennai, India,
ch.en.u4cys21026@ch.students.amrita.edu

Dinesh Kumar N
Department of Computer Science and Engineering,
Amrita Vishwa Vidyapeetham,
Chennai, India,
ch.en.u4cys21014@ch.students.amrita.edu

Abstract—In the rapidly evolving landscape of cybersecurity, the increasing complexity and frequency of cyber threats have rendered traditional perimeter-based security models insufficient. As enterprises move towards cloud-native applications and microservice based architectures, a more robust and granular security framework is required to address modern vulnerabilities. This paper introduces the design and implementation of a comprehensive Zero Trust Security Model for web applications running in Kubernetes Clusters. By utilizing key technologies such as JSON Web Token (JWT) authentication and mutual Transport Layer Security (mTLS), the proposed system ensures that no entity—whether internal or external—is trusted by default. Unlike traditional models, which rely on predefined trust zones, our approach enforces continuous identity verification, dynamic access control, and fine-grained micro-segmentation to safeguard sensitive data and application resources. The model incorporates several core Zero Trust principles, including encrypted communications, strict user and device authentication, and the application of real-time threat detection mechanisms. Furthermore, it includes robust policy enforcement to manage and protect APIs, containers, databases and other critical resources in a microservice based environment. Continuous monitoring of user behavior and the application of machine learning algorithms for threat detection bolster the system's ability to identify and mitigate potential security incidents in real time. Our experimental validation demonstrates the efficacy of this model in minimizing lateral movement attacks and unauthorized access, significantly reducing attack surfaces. The Zero Trust architecture we propose offers a scalable, secure, and resilient framework, making it well-suited to the demands of modern, containerized applications and ensuring the integrity and security of data flows across the application ecosystem.

Keywords— Zero Trust Security, JWT Authentication, Mutual TLS, Cybersecurity, Web Application Security, Adaptive Access Control, Micro-segmentation, Data Protection, Cloud-Based Architecture, Kubernetes Clusters.

I. INTRODUCTION

In today's digital landscape, organizations have faced an escalating number of cyber threats, which have highlighted the inadequacies of traditional perimeter-based security measures. The rapid adoption of agile methodologies, cloud computing, and containerization technologies has significantly expanded the attack surface, enabling cybercriminals to exploit vulnerabilities more effectively. Traditional perimeter-based security models, which rely on predefined trust zones, have struggled to keep pace with the

sophistication of modern distributed cyberattacks. To counteract these evolving threats, the Zero Trust Security Model has emerged as a transformative approach, fundamentally altering the way security is implemented across networks, applications, and devices. The Zero Trust Model operates on the principle that no entity—user, device, or application—should be trusted by default, and continuous verification is mandatory for every access attempt [1].

The Zero Trust Model addresses the shortcomings of traditional security frameworks by enforcing continuous authentication and authorization, regardless of the entity's origin within or outside the network [2]. Its fundamental principle, "never trust, always verify," mandates that access be granted based on a rigorous evaluation of predefined security policies. This approach is particularly relevant in microservice environments, where data and services are distributed across multiple containers and volumes. By utilizing technologies such as JSON Web Token (JWT) authentication, mutual TLS (mTLS), and micro-segmentation, Zero Trust strengthens security postures by limiting access to only authenticated and authorized entities. This paper aims to explore the design and implementation of a Zero Trust framework within a modern web application running in a microservice environment, addressing the limitations of traditional security models and proposing a more adaptive, resilient solution.

A) Analyzing Cybersecurity Dynamics within the Zero Trust Framework

The rise of cloud computing, distributed system and decentralized data has transformed how organizations operate, creating new challenges for cybersecurity. Traditional perimeter-based security approaches, often referred to as "castle-and-moat" defenses, assume that threats exist primarily outside the network perimeter, with internal resources inherently trusted.

However, the shift to remote work, cloud services, segmentation and decoupling of applications and data has made these boundaries porous, leaving applications vulnerable to insider threats, compromised resources, and increasingly sophisticated cyberattacks [3]. The Zero Trust Security Model represents a paradigm shift in how security is conceptualized. Unlike traditional models, Zero Trust does not rely on network location to establish trust. Instead,

it mandates that every entity—whether internal or external—be continuously authenticated and authorized before being granted access to sensitive resources as illustrated in Figure 1. The model leverages technologies like JWT for stateless authentication of the users and mTLS for secure, mutual verification of identity between containers or pods running in Kubernetes clusters ensuring that communication remains encrypted and secure [4]. This approach aligns with the decoupled nature of modern applications, which are segmented across multiple containers in Kubernetes clusters of various cloud providers like AWS and accessed by users and devices from varied locations.

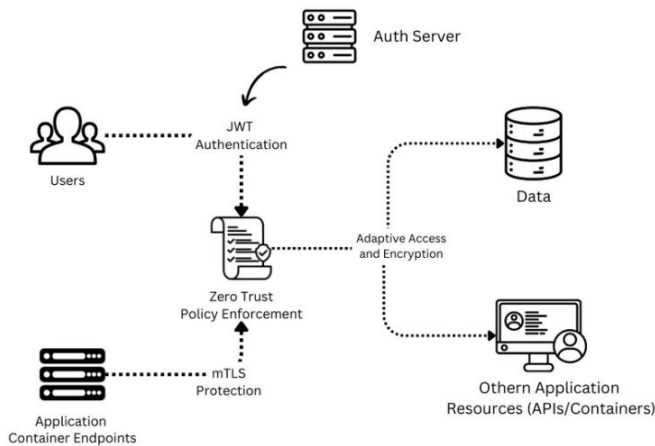


Fig. 1. Block diagram describing the design of ZTA

B) Problem Identification

The shift from perimeter-based security to Zero Trust is driven by several key challenges in securing modern microservice environments. Traditional security models often assume that threats are primarily external, leaving internal systems and data exposed once the perimeter is breached. This assumption results in several critical problems:

- **Expanded Attack Surface:** The adoption of agile, cloud computing, remote work, and decoupling practices has increased the usage of microservices, making it difficult to monitor and secure all potential entry points. Attackers can exploit vulnerabilities in containerized applications, or user credentials to gain unauthorized access.
- **Lateral Movement:** Once an attacker gains access to one pod in a Kubernetes cluster, they can move laterally across other containers without encountering significant restrictions, allowing for extensive damage of other pods and data breaches, as evidenced by several high-profile incidents.
- **Insufficient Identity Verification:** Traditional security models grant users and devices access based on network location, with minimal continuous verification. This lack of ongoing validation leaves Kubernetes clusters vulnerable to unauthorized access.
- **Distributed Systems Complexity:** As an application's, features and data are increasingly distributed across cloud platforms, implementing consistent security

policies to restrict and manage access control becomes more challenging.

The aforementioned challenges create a pressing need for a more robust, adaptable, and resilient security framework. The Zero Trust Security Model offers solutions by ensuring that all access requests are continuously authenticated, authorized, and validated, regardless of interservice or external communication.

C) Problem Statement

Traditional perimeter-based security models have proven inadequate for safeguarding sensitive data and resources in modern, microservice based environments. These environments, designed with a limited understanding of the threats facing today's decentralized networks, allow for lateral movement and insufficient identity verification, both of which leave Kubernetes clusters vulnerable to sophisticated cyberattacks. The Zero Trust Security Model addresses these challenges by implementing continuous identity verification, encrypted communication, and adaptive security policies across microservices, reducing the attack surface and preventing unauthorized access.

D) Significance and Motivation

Adopting the Zero Trust Security Model in modern web applications based on microservice architecture has become increasingly significant. The model addresses several critical issues facing organizations as they transition to cloud-native architectures, remote work models, and globally distributed operations across various cloud providers. The motivation for implementing Zero Trust in Kubernetes cluster lies in its ability to mitigate security threats inside the clusters, enhance protection for applications, reduce lateral movement, and meet regulatory compliance requirements. Mitigation of Internal Threats as Zero Trust enforces strict access controls and continuously verifies user identities, minimizing the risk of communicating containers.

Cloud Application Security as Zero Trust's dynamic approach to securing containerized environments ensures that access requests are evaluated in real time, protecting data across various platforms. Reduction of Lateral Movement by using micro-segmentation and enforcing stringent access control policies, Zero Trust limits an attacker's ability to move laterally within a compromised cluster [5]. As regulatory requirements intensify, organizations must demonstrate their ability to monitor and control access to sensitive data. Zero Trust provides the framework to meet these demands by ensuring continuous authentication and authorization of all access requests.

E) Objective of the Project

The primary objective of this project was to design and implement a secure microservice based web application that integrated the Zero Trust Security Model. The application incorporates JWT authentication, mTLS, and micro-segmentation and access control policies to provide a robust security framework. The goals were:

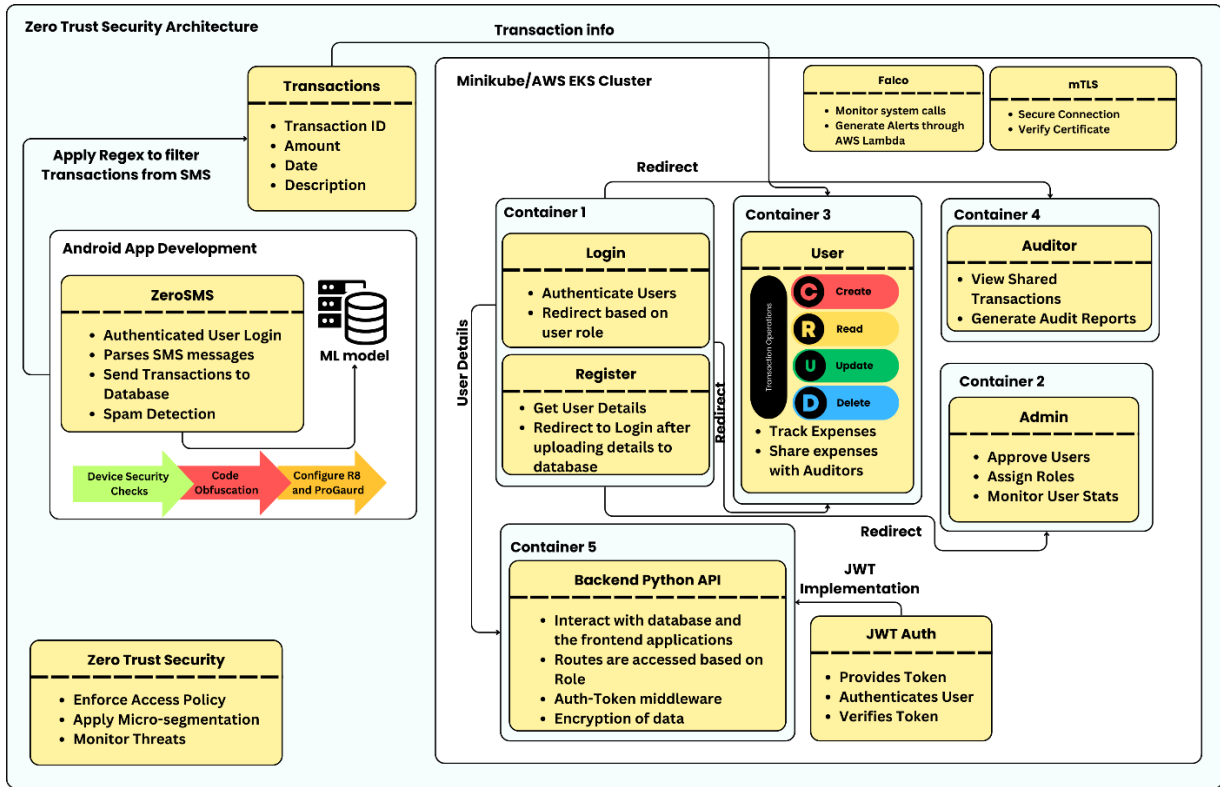


Fig. 2. Architecture diagram of the implementation of Zero Trust in the Web Application

- Continuous Identity Verification: Implement mechanisms to authenticate users and devices with every request, using JWT for stateless, token-based authentication and mTLS for mutual verification.
- Adaptive Access Control: Develop dynamic access control policies that adjust based on user roles and request context, ensuring minimal privilege for users.
- Encrypted Communication: Secure all communications within the application using mTLS to protect against interception and tampering.
- Prevention of Lateral Movement: Implement micro-segmentation to isolate components and prevent attackers from moving laterally across the system.
- Real-Time Threat Detection: Integrate machine learning algorithms to detect anomalies in user behavior, allowing for proactive security measures [6].

II. RELATED WORKS

D'Silva et al. examined the implementation of Zero Trust Architecture (ZTA) using Kubernetes [7], highlighting its significance in cloud-based environments. They emphasized how ZTA continuously verified users, devices, and applications, challenging traditional trust models. The study discussed the use of technologies such as Kubernetes, Docker and RBAC/ABAC for enhanced access control. Additionally, they critiqued traditional security frameworks, noting the inadequacies of perimeter-based models in modern cloud infrastructures. Their proposed architecture improved security through continuous verification and logging, making it adaptable to decentralized systems.

Varalakshmi et al. [8] explored the enhancement of JSON Web Token (JWT) authentication within Software Defined Networks (SDNs). They identified vulnerabilities in traditional JWT implementations and proposed a modified authentication approach that improves security without

sacrificing performance. Their analysis highlighted the importance of securing API endpoints and user sessions, ultimately enhancing the overall integrity of network communications in SDN environments. Bucko et al. [9] examined the enhancement of JWT authentication and authorization in web applications by utilizing user behavior history. Their study analyzed how behavior-based metrics could improve the reliability of authentication processes. They proposed a dynamic authentication mechanism that adapts based on user activity, significantly reducing unauthorized access and enhancing user security in web applications.

Jánoky et al. [10] conducted an analysis of revocation mechanisms for JSON Web Tokens (JWTs). They identified the challenges associated with effectively revoking tokens without degrading system performance. Their research emphasized the need for efficient revocation strategies to ensure the security of applications relying on JWTs, contributing valuable insights into the management of token lifecycles. Achary and Shelke [11] investigated fraud detection in banking transactions using machine learning techniques. They presented a framework that leveraged various algorithms to identify and mitigate fraudulent activities in real-time. Their findings underscored the efficacy of machine learning in enhancing the security of financial transactions, providing banks with tools to protect against fraud. The authors highlighted the importance of adaptive models that can learn from evolving fraud patterns, contributing to more robust credit card security measures.

Fraudulent Transaction Detection in Credit Card by Applying Ensemble Machine Learning Techniques by Prusti et al. [12] explores the use of ensemble machine learning models for detecting fraudulent credit card transactions. By combining multiple machine learning algorithms, the study aims to improve accuracy, precision,

and detection speed. Key findings show that ensemble methods outperform individual classifiers in identifying fraudulent transactions, reducing false positives, and increasing overall efficiency in real-time fraud detection systems. Jacqueline Priya and Saradha [13] reviewed machine learning algorithms for fraud detection and prevention. Their comprehensive analysis encompassed various techniques, assessing their strengths and weaknesses in different contexts. The paper provided a critical overview of the current state of fraud detection systems, emphasizing the potential of machine learning in enhancing predictive accuracy and operational efficiency in fraud management.

A. I. Weinberg et al. [14] studied Zero Trust Architecture (ZTA) for applications and network security, highlighting its relevance in contemporary cybersecurity frameworks. The research detailed how ZTA eliminates implicit trust and enforces strict access controls, providing a robust model for securing sensitive data in dynamic environments. Kang et al. [15] provided a brief survey of the theory and application of Zero Trust security. They examined various implementations and discussed how Zero Trust principles could enhance overall security posture. Their findings illustrated the applicability of Zero Trust in diverse settings, emphasizing its importance in mitigating modern cybersecurity threats.

Ashfaq et al. [16] presented a comprehensive survey of the Zero Trust security paradigm. Their research analyzed existing literature, outlining key principles and strategies for implementing Zero Trust in various contexts. The authors identified gaps in current research, suggesting areas for future investigation to enhance understanding and application of Zero Trust principles. Liu et al. [17] dissected the research landscape surrounding Zero Trust and its implementation in the Internet of Things (IoT). They identified key challenges

and strategies for applying Zero Trust principles in IoT environments, emphasizing the necessity for stringent access controls and continuous verification to mitigate security risks. The authors provided a comprehensive overview of current research trends and highlighted areas requiring further investigation to enhance IoT security through Zero Trust frameworks.

Mehraj and Banday [18] proposed a Zero Trust strategy for cloud computing environments to address security challenges such as identity theft, data breaches, and trust management issues. Their model emphasizes strong access control and continuous verification due to the dynamic, shared nature of cloud services. The authors discussed how traditional security approaches are insufficient in cloud deployments and highlighted the Importance of Zero Trust in tracking and mitigating both external and insider threats.

Muddinagiri et al. [19] presented a method for deploying Kubernetes locally using Minikube to manage Docker containers, emphasizing the advantages of containerization as a lightweight alternative to virtual machines. The paper highlights the importance of local Kubernetes testing for industries like finance and healthcare, which require secure, scalable applications without relying on cloud-based deployments. Their approach was demonstrated using a Python-based web server built with a DockerFile. Pace's thesis [20] focuses on the implementation of Zero Trust Networks using Istio in Kubernetes environments. Istio, as a service mesh, manages traffic, observability, and security, offering features like mutual TLS, JWT-based authentication, and seamless integration with Kubernetes, enhancing microservices security and operational efficiency

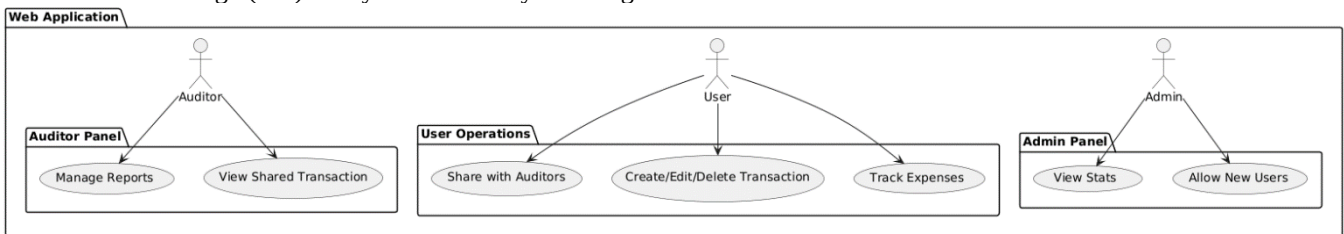


Fig. 3. Activity Diagram for the Web Application using RBAC

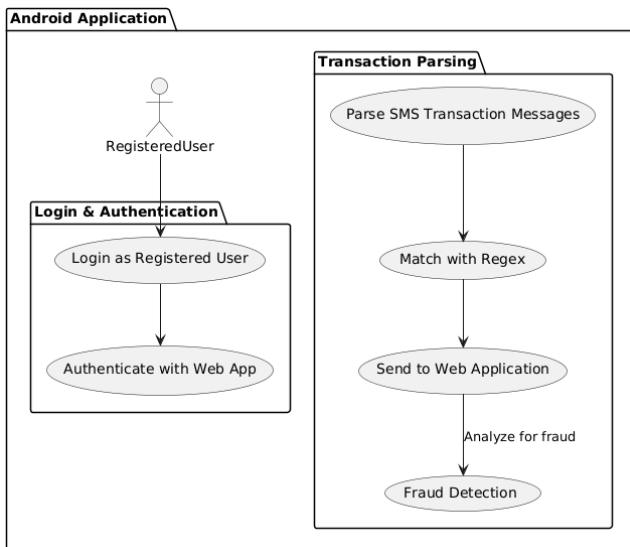


Fig. 4. Activity Diagram for the ZeroSMS Android Application

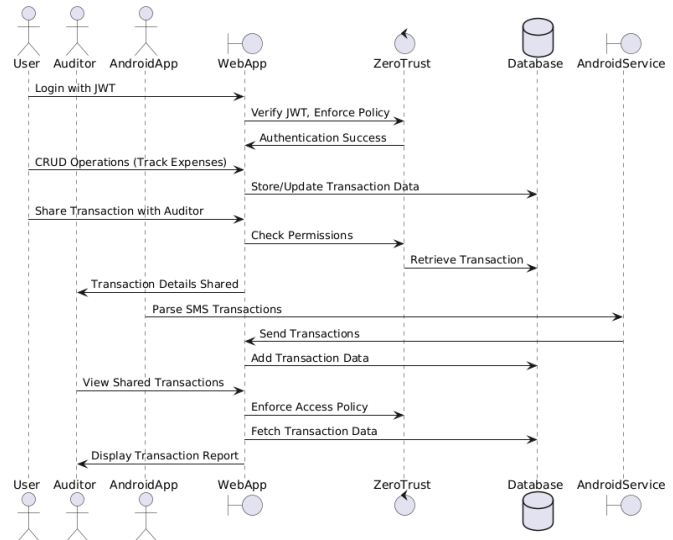


Fig. 5. Sequence Diagram representing the order of events taking place

III. Methodology

The proposed containerized web application is designed to provide users with a comprehensive expense tracking system that adheres to Zero Trust security principles as illustrated in Figure 2. Built using React.js for the frontend, styled with Tailwind CSS, and supported by a MongoDB backend with a Python API, the application allows users to manage their financial transactions efficiently while ensuring secure access and data integrity at all levels. This application uses Docker for containerization, Kubernetes for Container Orchestration and Helm to dynamically handle configurations of Kubernetes manifests. The complete application is then deployed to AWS EKS cluster remotely enforced with strict IAM policies and VPC.

Web Application Overview

At the core of this application is the user interface, which enables users to perform CRUD operations (Create, Read, Update, Delete) on their financial transactions. Users can manually input their expenses, savings, and investments through a form-based system that adds each entry as a card component. Additionally, users can visualize their transaction data through graphical views implemented using react-chart.js, providing insights into spending patterns and financial health over time. A distinguishing feature of this application is its integration with the ZeroSMS mobile app. The ZeroSMS app automatically parses SMS messages related to transactions, which are often sent by banks or payment services, and synchronizes them with the web application. This functionality is especially beneficial for users who may forget to log some expenses or are too busy to manually enter transaction details. Once an SMS is parsed, the web application notifies the user, allowing them to classify and manage these transactions at their convenience. This automated process helps ensure that even small purchases or overlooked transactions are accounted for within the expense tracker.

Another key feature is the ability for users to share their transaction data with a selected list of auditors. The auditors, who are granted read-only access to user transactions, can view the data to prepare financial reports. This feature is essential for those who require external auditing services or want to maintain transparency with financial advisors. The auditor's interface displays a list of users who have shared their transaction data, allowing them to access the data without modifying it, which aligns with Zero Trust's principle of minimal privileges for authorized users.

Admin and Auditor Interfaces

The admin interface is critical for managing user roles and access within the application. New users must first register through a registration page, but their access to the system is only granted upon approval by the admin. The admin has the authority to approve or reject users, as well as assign specific roles such as user, auditor, or admin. This role assignment functionality is presented as a table view, where the admin can easily manage user permissions. Role-based access control (RBAC) ensures that each user only has access to the resources necessary for their role, supporting the Zero Trust model's principle of least privilege. In addition to user management, the admin interface provides a user behavior analysis dashboard. This feature enables the admin to monitor important logs, such as user login and logout times, system activity, and other behaviors that may indicate suspicious activity. By continuously monitoring these behaviors, the application can detect anomalies,

helping to prevent unauthorized access or potential misuse of the system. This feature reinforces Zero Trust's "never trust, always verify" approach, where users are continuously authenticated and their behavior is scrutinized to detect potential threats.

Security and Fraud Detection

The security architecture of the application is built on the Zero Trust framework, which ensures that each user and each transaction is treated as potentially untrusted until verified. Each request to access sensitive data is validated JSON Web Tokens (JWT), which provide secure, time-bound access tokens that validate the user's identity and role. If a user's JWT token is invalid or expired, access is immediately denied, ensuring that no unauthorized requests are processed. The fraud detection model is another critical component of the system. Developed using machine learning (ML) algorithms, the model analyzes transaction patterns to detect potentially fraudulent activity. It is trained on spam SMS dataset from Kaggle, leveraging algorithms such as random forest and support vector machines (SVM) to classify transactions as legitimate or suspicious. This real-time analysis is particularly valuable for auditors, as they can identify anomalies or red flags that may require further investigation. The fraud detection model complements the Zero Trust approach by providing continuous monitoring and detection capabilities, ensuring that even authorized users do not engage in fraudulent behavior. The application architecture is illustrated in Figure 3 and Figure 4, which shows the flow of user and admin interactions, SMS parsing through ZeroSMS, and fraud detection. Figure 5 illustrates the user behavior analysis dashboard, showing how logs and user activity are tracked and monitored in real-time.

Access Tokens functionality and Implementation

In the context of Zero Trust security architecture, the utilization of Access Tokens, specifically JSON Web Tokens (JWTs), is essential for maintaining robust authentication and authorization mechanisms. JWTs are compact, URL-safe tokens that enable secure information exchange between parties. Comprising three parts—the header, which defines the signing algorithm; the payload, which contains user claims; and the signature, used to verify the token's authenticity—JWTs provide a stateless solution that enhances both scalability and security in distributed systems. This structure allows for seamless authentication flows, where the server can validate the token without maintaining session information.

The Python APIs to generate and manage and rigorously validate JWT are containerized and built to run as a separate pod inside the Kubernetes cluster which ensures continuous authentication of requests from the clients. Upon successful authentication, the system generates a JWT containing essential user-specific claims, such as roles and permissions, which is then transmitted to the user. For subsequent requests, the user includes this token in the authorization header, allowing the application to verify the token's integrity and validity before granting access to protected resources. This implementation not only adheres to the principle of least privilege by limiting access based on the user's current permissions but also enhances security through the use of short-lived access tokens and refresh tokens, which mitigate the risks associated with token theft or misuse. The lifecycle of JWTs, including issuance, expiration, and renewal mechanisms, is depicted in Figure. 6.

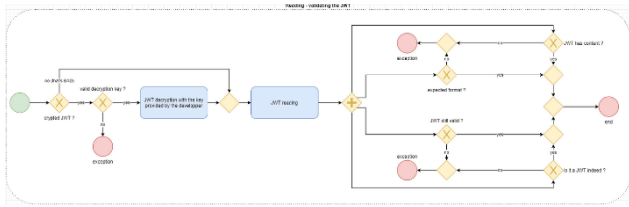


Fig. 6. JWT Life Cycle

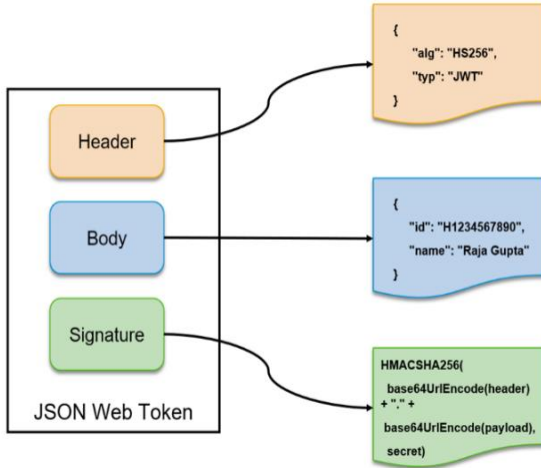


Fig. 7. JWT Architecture

To fortify the security of JWT implementations, organizations can integrate additional security measures such as Multi-Factor Authentication (MFA) and continuous monitoring. By assessing user behavior, geolocation, and device health alongside JWT validation, a comprehensive security posture is achieved that aligns with the core principles of Zero Trust. Furthermore, effective revocation mechanisms for JWTs are crucial to ensure that compromised tokens are invalidated promptly. The implementation of these mechanisms, such as utilizing a token blacklist or short-lived tokens with refresh capabilities, is vital for maintaining a secure environment in Zero Trust applications. Overall, the strategic integration of JWTs within Zero Trust frameworks not only streamlines access management but also ensures a continuous verification process that upholds the security integrity of the application.

SMS Parsing and Spam Detection in Financial Transactions

The increasing reliance on digital financial transactions has highlighted the need for comprehensive expense management solutions that ensure users capture all transactions accurately. The ZeroSMS app addresses this challenge by parsing SMS messages related to financial transactions and classifying them as either transaction-related or non-transaction-related. This functionality not only helps users track their expenses more effectively but also mitigates the risk of overlooking minor transactions such as small purchases and pocket money. By integrating the ZeroSMS app with a web application, users can manage their finances in a centralized location while also ensuring that any missed transactions are flagged for review.

Once the SMS messages are parsed, the app identifies those that pertain to transactions and sends them to the web application, where they appear in a notification center. Users can then revisit these notifications to categorize the transactions as expenses, savings, or investments, and to provide additional information as needed. This approach enhances the accuracy of expense tracking and ensures that all relevant financial activities are recorded. As illustrated

in Figure. 8, the workflow demonstrates how SMS parsing feeds into the user's expense management process, emphasizing the seamless integration between the mobile and web platforms.

To further enhance the functionality of the ZeroSMS app, a Machine Learning (ML) model was developed to detect fraudulent transactions. The model was built using a combination of supervised learning techniques and ensemble methods, specifically leveraging algorithms such as Random Forest, Gradient Boosting, and Support Vector Machines (SVM). A diverse dataset comprising historical transaction data, including both legitimate and fraudulent transactions, was utilized to train the model. Feature engineering played a critical role in this process, where features such as transaction amount, frequency, and user behavior patterns were extracted from the data to improve classification accuracy.

The ML model underwent several phases of development, including data preprocessing, model training, and validation. During preprocessing, the data was cleaned and normalized, addressing any inconsistencies or missing values. A cross-validation technique was employed to evaluate the model's performance, ensuring that it generalized well to unseen data. After extensive testing, the model achieved an accuracy of over 96%, demonstrating its effectiveness in identifying potential fraud. As shown Table 1, the performance metrics highlight the model's precision and recall rates, emphasizing its capability to minimize false positives and negatives in spam detection

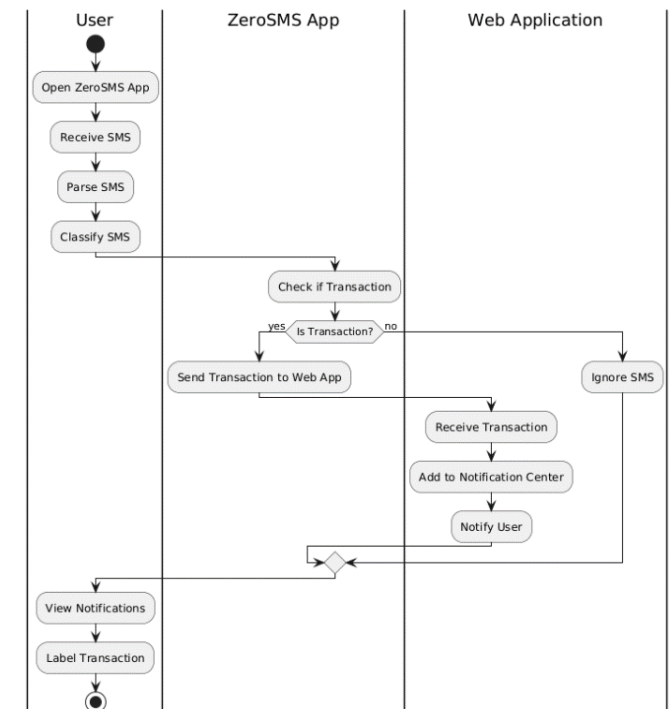


Fig. 8. Algorithm of the interaction between App and Web App

The integration of this ML model within the ZeroSMS app allows for real-time monitoring of transactions, alerting users to any suspicious activities based on predefined thresholds and user behavior patterns. By continuously learning from new data, the model can adapt to evolving fraudulent strategies, ensuring that users are protected against financial fraud. Ultimately, the ZeroSMS app not only aids in managing expenses but also provides a robust layer of security against fraudulent transactions, enhancing users' overall financial management experience.

Policy Enforcement Point for Role Based Access Control

A Policy Enforcement Point (PEP) is a crucial component in enforcing security within web applications, especially in the context of a Zero Trust Architecture (ZTA). The PEP acts as a gatekeeper that intercepts all incoming requests before they reach the application's core services, ensuring that only authorized users or services gain access. When implemented in microservice based web applications, the PEP evaluates each request based on predefined security policies. These policies determine who can access which service and resources, at what time, and under what conditions. The PEP typically works alongside a Policy Decision Point (PDP), which makes the decision regarding access based on rules such as role-based access control (RBAC) or attribute-based access control (ABAC) as shown in Figure 3. Once a decision is made by the PDP, the PEP either grants or denies access.

In practical terms, the PEP is embedded at critical points within the microservice architecture, such as API gateways or middleware layers. Every incoming request to the service of application containers passes through the PEP, where it is authenticated, and authorization policies are applied. If the user or service is unauthorized or violates policies, the PEP blocks access. For an instance, in our application a, the PEP ensures that an auditor is allowed to access only the transactions and data related to their client. Also, the routes behind the services are restricted and are specific to the entities (user, admin, auditor) to access. It could also enforce least privilege access, allowing users or services to only perform the actions necessary for their role.

Additionally, PEPs can be deployed as sidecar containers in microservices, ensuring that access controls are enforced at each service endpoint. This is especially important when services communicate internally within distributed environments. Overall, PEP strengthens security by ensuring consistent policy enforcement, providing granular access control, and mitigating potential attack vectors.

mTLS to secure communication within the Kubernetes Clusters

Mutual TLS (mTLS) is a foundational security mechanism in implementing Zero Trust Architecture (ZTA) for microservices [21]. It provides robust authentication and encrypted communication between microservices by requiring both the client and server to present valid certificates, ensuring mutual trust before communication occurs. This mutual verification process is key to the “never trust, always verify” principle of ZTA.

In mTLS, each pod in the cluster is assigned a unique cryptographic certificate issued by a trusted Certificate Authority (CA). When two services communicate, both sides (client and server) present their certificates during the handshake process. Each service verifies the other's certificate against the CA's public key to ensure authenticity. Once both certificates are validated, an encrypted communication channel is established using TLS (Transport Layer Security). This ensures that data exchanged between services is not only authenticated but also protected from eavesdropping and tampering. mTLS is integrated into our application service meshes using a tool named *Istio* which automate certificate management and enforce mTLS at the network layer. In these environments, mTLS is applied transparently to microservices, without requiring each service to implement its own TLS logic. The

service mesh handles certificate issuance, renewal, and rotation, ensuring seamless mTLS enforcement.

Unlike traditional systems that assume trust once inside the network, mTLS requires that every interaction between services involves identity verification [22]. This eliminates the risk of unauthorized services accessing resources, as only those with valid certificates are allowed to communicate. When combined with Policy Enforcement Points (PEPs), mTLS strengthens the overall ZTA. The PEP verifies that requests originate from authorized services with valid certificates before granting access to resources.

IV. ALGORITHM

```
1: ZeroTrustArchitecture main
2: Initialize ()
3: if UserLogin then
4:   user ← AuthenticateUser()
5:   if user.authorized then EnforceLeastPrivilege(user)
6:   else DenyAccess()
7: end if
8: end if
9: if RequestAccessToResource then
10:
11:   if VerifyAccess(user) then LogAccess(user)
12:   else DenyAccess()
13: end if
14: end if
15: if ManageContainers then
16:   InitializeAndSecureKubernetes()
17:   DeployContainersWithmTLS()
18: end if
19: if DockerizeApp then BuildDeployAndSecureApp()
20: end if
21: if MonitorActivity then AuditAndDetectThreats(user)
22: end if
23: end procedure
24: procedure AuthenticateUser
25:   return ValidateCredentialsAndAssignRole()
26: end procedure
27: procedure VerifyAccess(user)
28:   return ValidateTokenAndAccess(user)
29: end procedure
```

V. RESULTS

The implementation of the Zero Trust application successfully demonstrated the core principles of Zero Trust Architecture (ZTA) by ensuring strict access control, continuous authentication, and verification of users and devices. The system effectively secured sensitive resources by integrating role-based access control (RBAC) and least-privilege policies. Performance benchmarks showed that the dynamic policy enforcement, managed via Kubernetes, allowed for efficient assignment of user roles (public user, user, admin, auditor) based on JWT authentication. Furthermore, network traffic was consistently monitored and verified before granting access to internal services, which significantly mitigated risks associated with unauthorized access.

The integration of Kubernetes for container orchestration further enhanced scalability and resource management, allowing the system to adapt to the changing security requirements in real-time. Overall, the implementation exhibited robust security postures with minimal performance overhead.

Table 1. Comparison of Model Performance Metrics

Model	F1 Score	Recall	Precision	Accuracy	Mean Squared Error	ROC AUC Score	Mean Absolute Error	Specificity
Naive Bayes	0.918	0.918	0.918	0.978	0.022	0.975	0.022	0.988
SVM	0.9505	0.826	1.0	0.977	0.023	0.984	0.023	1.0
Logistic Regression	0.909	0.845	0.984	0.978	0.022	0.983	0.022	0.998
Random Forest	0.905	0.826	1.0	0.977	0.023	0.981	0.023	1.0
Gradient Boosting	0.87	0.781	0.983	0.969	0.031	0.973	0.031	0.998

The web application was deployed in an EKS cluster, utilizing node groups belonging to the EKS Cluster. As illustrated in Figure 9., the architecture ensures scalability and fault tolerance, with seamless communication between services for a smooth user experience.

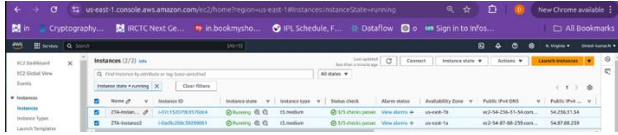


Fig 9. Node Groups of EKS Cluster deployed as EC2 instances

Security within the application is strengthened through AWS Lambda and the Falco tool, which monitors the cluster for anomalies and triggers alerts for real-time threat detection, as illustrated in Figure 10. showcases the alerts from Lambda as logs captured in AWS Cloudwatch. These logs contain valuable information such as the violated rule, container name, container ID, pod name, and namespace where the attack occurred, along with the system call used to perform the attack. This collective data emphasizes the implementation of the Zero Trust strategy and aids in identifying the root cause of suspicious activity. Additionally, session logs provide a thorough audit trail, tracking user activities and identifying suspicious behavior. These logs ensure compliance with Zero Trust principles by ensuring all user actions are continuously monitored and recorded.

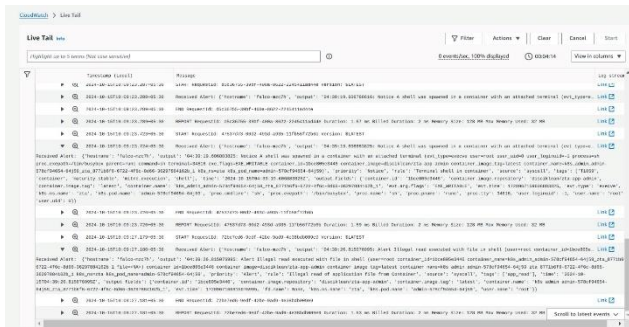


Fig 10. AWS Lambda Alerts triggered by Falco tool deployed in the cluster

As far as the Spam detection model is considered, Naive Bayes was the best model with an F1 score of 0.918 and accuracy of 0.978 as shown in the Figure 11 and Table 1, showing a strong balance of precision and recall. SVM had the highest precision (1.0) and ROC AUC score (0.984), while Logistic Regression also performed well with an F1 score of 0.909. Random Forest and Gradient Boosting had slightly lower F1 scores but maintained high accuracy across all models.

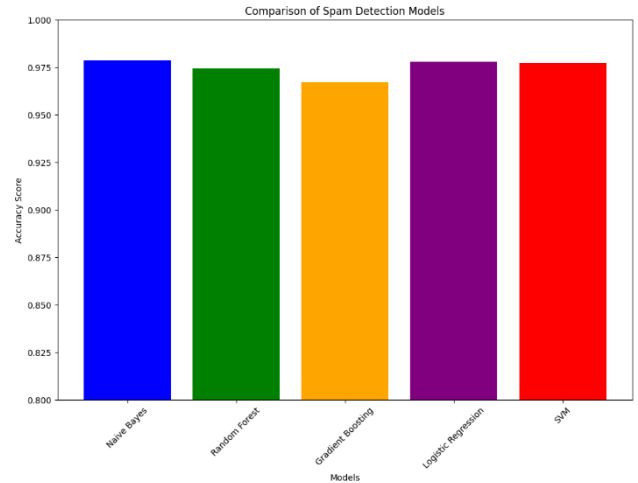


Fig. 11. Comparison between Spam Detection Models

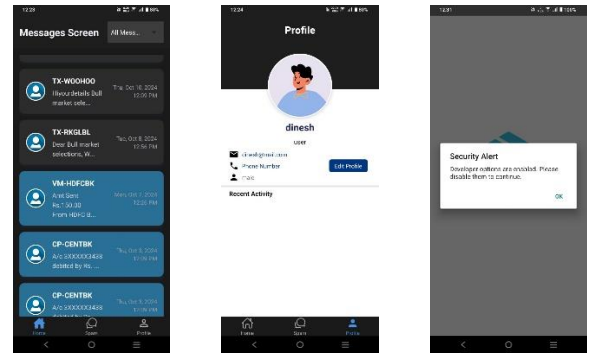


Fig. 12. ZeroSMS in production release with security features

To prepare for production release, the app leverages the JailMonkey library to detect rooting and unsafe environments, while utilizing R8 and ProGuard tools for code optimization and obfuscation to enhance performance and security as shown in Figure 12. The APK is securely signed with a private key from a trusted keystore, ensuring authorized distribution. Figure 13. and Figure 14. compares and tests ZeroSMS APK against standard APKs. Due to R8 and ProGuard obfuscation, reverse engineering ZeroSMS is significantly more challenging. The MobSF scan illustrated in Figures 15 a) & b) shows ZeroSMS achieving a security score of 57 (Grade B), 83.87% higher than a standard APK, which scored 31 (Grade C). ZeroSMS has SMS permissions and allows "http" traffic for backend connectivity, impacting its security score. In contrast, the standard APK is merely a "Hello World" application built in Android Studio, lacking any additional privileges.

```

Download file
package com.example.firstapp;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    Button add;
    TextView display;
    Button reset;
    Button sub;
    EditText x;
    EditText y;

    /* access modifiers changed from: protected */
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        this.x = (EditText) findViewById(R.id.et1);
        this.y = (EditText) findViewById(R.id.et2);
        this.add = (Button) findViewById(R.id.b1);
        this.sub = (Button) findViewById(R.id.b2);
        this.reset = (Button) findViewById(R.id.b3);
        this.display = (TextView) findViewById(R.id.tv);
        this.add.setOnClickListener(new View.OnClickListener() {
            public void onClick(View v) {
                MainActivity.this.display.setText(Integer.toString(Integer.parseInt(MainActivity.this.x.getText().toString()) + Integer.parseInt(MainActivity.this.y.getText().toString()));
            }
        });
        this.sub.setOnClickListener(new View.OnClickListener() {
            public void onClick(View v) {
                MainActivity.this.display.setText(Integer.toString(Integer.parseInt(MainActivity.this.x.getText().toString()) - Integer.parseInt(MainActivity.this.y.getText().toString()));
            }
        });
        this.reset.setOnClickListener(new View.OnClickListener() {
            public void onClick(View v) {
                MainActivity.this.x.setText("");
                MainActivity.this.y.setText("");
                MainActivity.this.display.setText("");
            }
        });
    }
}

```

Fig. 13 Reverse Engineering of Standard APKs

```

ZeroSMS.apk
Download ZIP
ZeroSMS.apk / sources / e1 / C0522a.java

Download file
package e1;
import d3.A;
import d3.C051a;

/* renamed from: e1.a reason: case insensitive filesystem */
public abstract class C0522a {
    public static void a(A a, Object obj) {
        for (C051a aVar : a.c().a()) {
            if (obj.equals(aVar.a().a())) {
                aVar.cancel();
                return;
            }
        }
        for (C051a aVar2 : a.c().a()) {
            if (obj.equals(aVar2.a().a())) {
                aVar2.cancel();
                return;
            }
        }
    }
}

```

Fig. 14 ZeroSMS with class name and code Obfuscation

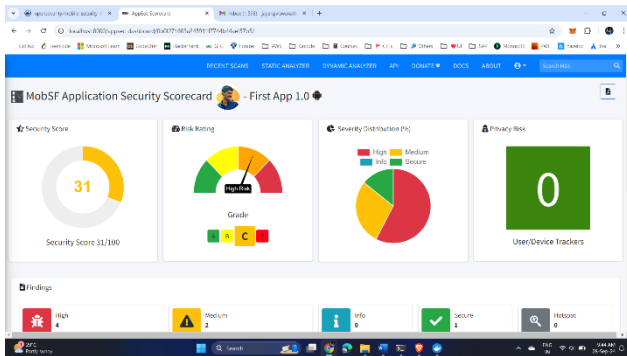


Fig. 15. a) MobSF scan result of standard APK

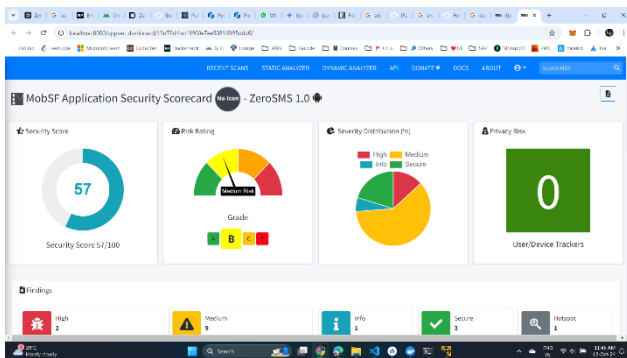


Fig. 15. b) MobSF scan result of ZeroSMS

VI. CONCLUSION AND FUTURE WORK

The implementation of Zero Trust Architecture (ZTA) represents a paradigm shift in cybersecurity strategies, particularly in the context of cloud-based environments and decentralized systems. The traditional perimeter-based security models have proven inadequate in the face of increasingly sophisticated cyber threats and the complexities introduced by remote work and cloud computing. By rejecting the notion of implicit trust and emphasizing continuous verification of users and devices, ZTA addresses the vulnerabilities inherent in legacy security frameworks. The integration of advanced technologies, such as Kubernetes and JSON Web Tokens (JWT), plays a crucial role in facilitating the deployment of

Zero Trust principles. Studies have shown that leveraging these technologies can enhance security posture significantly by enabling fine-grained access control and dynamic policy enforcement. Moreover, the continuous logging and monitoring of user actions contribute to an organization's ability to detect and respond to potential threats promptly. This proactive approach not only safeguards sensitive data but also fosters a culture of security awareness within organizations. As organizations continue to embrace digital transformation, the adoption of Zero Trust models will be essential for mitigating risks associated with modern computing environments. Future research should focus on addressing the challenges of implementing ZTA in various contexts, including the Internet of Things (IoT) and decentralized identity management. By refining these frameworks and exploring innovative solutions, cybersecurity professionals can better protect against evolving threats and ensure the resilience of their systems in an increasingly interconnected world.

Future enhancements to this Zero Trust application could include more advanced behavioral analytics to detect anomalous user activities and improve response to insider threats. Integrating machine learning algorithms for adaptive risk scoring would allow the system to dynamically adjust access privileges based on real-time user behavior. Additionally, extending the system's scope to integrate with external identity providers through federated identity management and supporting hybrid or multi-cloud environments could further strengthen security. Implementing decentralized identity management using blockchain or Distributed Ledger Technology (DLT) could provide greater transparency and security for identity verification processes. Expanding the architecture to include security information and event management (SIEM) systems for enhanced logging, monitoring, and real-time alerts would also be a valuable addition to improve threat detection and incident response times.

VII. REFERENCES

- [1] O. E. Imokhai, T. E. Emmanuel, A. A. Joshua, E. S. Emakhu, and S. Adebisi, "Zero Trust Architecture: Trend and Impact on Information Security," *International Journal of Emerging Technology and Advanced Engineering*, vol. 12, no. 7, pp. 87-92, July 2022.
- [2] S. Rose, O. Borchert, S. Mitchell, and S. Connolly, *NIST Special Publication 800-207 Zero Trust Architecture*. Gaithersburg, MD: National Institute of Standards and Technology (NIST), 2020.
- [3] Y. He, D. Huang, L. Chen, Y. Ni, and X. Ma, "A Survey on Zero Trust Architecture: Challenges and Future Trends," *Wireless Communications and Mobile Computing*, vol. 2022, Article ID 6476274, 13 pages, 2022. DOI: <https://doi.org/10.1155/2022/6476274>.
- [4] A. Mustyala and S. Tatineni, "Advanced Security Mechanisms in Kubernetes: Isolation and Access Control Strategies," *ESP Journal of Engineering & Technology Advancements*, vol. 1, no. 2, pp. 45-52, 2021.
- [5] B. Pranoto, "Threat Mitigation in Containerized Environments," *Applied Research in Artificial Intelligence and Cloud Computing*, vol. 6, no. 8, pp. 142-151, 2023.
- [6] S. Bagheri, H. Kermabon-Bobinnec, S. Majumdar, Y. Jarraya, L. Wang, and M. Pourzandi, "Warping the Defence Timeline: Non-disruptive Proactive Attack Mitigation for Kubernetes Clusters," in *Proc. IEEE International Conference on Communications (ICC 2023)*, Rome, Italy, 28 May - 01 June, 2023, pp. 567-574.
- [7] D. D'Silva and D. D. Ambawade, "Building a Zero Trust Architecture Using Kubernetes," in *Proc. 2021 6th International Conference for Convergence in Technology (I2CT)*, Pune, India, Apr. 2021, pp. 1-5.
- [8] P. Varalakshmi, B. Guhan, P. V. Siva, T. Dhanush, and K. Saktheeswaran, "Improvising JSON Web Token Authentication in SDN," in *Proc. 2022 International Conference on Communication*,

Computing and Internet of Things (IC3IoT), Mar. 2022. DOI: 10.1109/IC3IOT53935.2022.9767873.

- [9] A. Bucko, K. Vishi, B. Krasniqi, and B. Rexha, "Enhancing JWT Authentication and Authorization in Web Applications Based on User Behavior History," *Computers*, vol. 12, no. 4, pp. 78-85, 2023. DOI: <https://doi.org/10.3390/computers12040078>.
- [10] L. V. Jánoky, J. Levendovszky, and P. Ekler, "An Analysis on the Revoking Mechanisms for JSON Web Tokens," *International Journal of Distributed Sensor Networks*, vol. 14, no. 9, Aug. 2018. DOI: 10.1177/1550147718801535.
- [11] R. Achary and C. J. Shelke, "Fraud Detection in Banking Transactions Using Machine Learning," in *Proc. 2023 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)*, Jan. 2023.
- [12] D. Prusti and S. K. Rath, "Fraudulent Transaction Detection in Credit Card by Applying Ensemble Machine Learning Techniques," in *Proc. Jun 2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*,
- [13] G. J. Priya and S. Saradha, "Fraud Detection and Prevention Using Machine Learning Algorithms: A Review," in *Proc. 2021 7th International Conference on Electrical Energy Systems (ICEES)*, Feb. 2021. DOI: 10.1109/ICEES51510.2021.9383631.
- [14] A. I. Weinberg and K. Cohen, "Zero Trust Implementation in the Emerging Technologies Era: A Survey," *Complex Engineering Systems*, vol. 4, pp. 16-28, 2024. DOI: <http://dx.doi.org/10.20517/ces.2024.41>.
- [15] H. Kang, G. Liu, Q. Wang, L. Meng, and J. Liu, "Theory and Application of Zero Trust Security: A Brief Survey," *Entropy*, vol. 25, no. 12, 2023.
- [16] S. Ashfaq, S. A. Patil, S. Borde, P. Chandre, and P. M. Shafi, "Zero Trust Security Paradigm: A Comprehensive Survey and Research Analysis," *Journal of Electrical Systems*, vol. 19, no. 2, pp. 28-37, 2023.
- [17] C. Liu, R. Tan, Y. Wu, Y. Feng, Z. Jin, F. Zhang, Y. Liu, and Q. Liu, "Dissecting Zero Trust: Research Landscape and Its Implementation in IoT," *Cybersecurity*, vol. 7, no. 20, pp. 1-10, 2024. DOI: <https://doi.org/10.1186/s42400-024-00212-0>.
- [18] S. Mehraj and M. T. Banday, "Establishing a Zero Trust Strategy in Cloud Computing Environment," in *Proc. 2020 International Conference on Computer Communication and Informatics (ICCCI 2020)*, Coimbatore, India, Jan. 2020.
- [19] R. Muddinagiri, S. Ambavane, and S. Bayas, "Self-Hosted Kubernetes: Deploying Docker Containers Locally with Minikube," in *Proc. 2019 International Conference on Innovative Trends and Advances in Engineering and Technology (ICITAET)*.
- [20] M. Pace, *Zero Trust Networks with Istio*, Master's thesis, Politecnico di Torino, Turin, Italy, 2020.
- [21] A. Kurbatov, *Design and Implementation of Secure Communication Between Microservices*, Master's thesis, Aalto University, Espoo, Finland, 2020.
- [22] Y. Yang, W. Shen, B. Ruan, W. Liu, and K. Ren, "Security Challenges in the Container Cloud," in *Proc. 2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, Dec. 2021. DOI: 10.1109/TPS-ISA52974.2021.